



**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS
ACADEMY (ICDFA)**

SBT-DF202 · PRACTICAL LABORATORIES · LAB 4
STEGANOGRAPHY AND HIDDEN DATA DETECTION

STUDENT NAME: IBRAHIM ISHAKU
STUDENT ID: 2025/FWSD/11334

**FELLOWSHIP IN WEB APPLICATION SECURITY & DIGITAL
FORENSICS**

COURSE: BASIC COMPUTER SKILLS FOR DIGITAL FORENSICS
STEGANOGRAPHY AND HIDDEN DATA DETECTION

INSTRUCTOR: AMINU IDRIS, AMCPN

5TH SEPTEMBER, 2026



Acknowledgement

I thank my instructor for the comprehensive Steganography and Hidden Data Detection materials that formed the foundation for this lab. The practical walkthrough on Steghide, LSB substitution, steganography vs encryption, and password recovery gave me the essential skills required for this practical exercise. I also appreciate the International Cybersecurity and Digital Forensics Academy (ICDFA) for providing the lab environment and carrier image for this practical assessment.



Certification / Declaration

I, **Ibrahim Ishaku**, with Student ID: **2025/FWSD/11334**, hereby declare that this lab report titled "**Steganography and Hidden Data Detection**" is my original work. All steganography embedding, extraction, hash verification, and password recovery tasks documented in this report were performed by me in the ICDFA Kali Linux lab environment. The answers are based on my actual practical work and understanding of the course materials. I confirm that the original carrier image was preserved and that all stego images, extracted files, and wordlists were documented with cryptographic hashes.

Student Signature: _____

Date: 5th September, 2026



Instructor Approval Page

Instructor Name:

Comments / Evaluation:

Instructor Signature: _____

Date: _____



Table of Contents

Section	Page
Cover Page	i
Acknowledgement	ii
Certification / Declaration	iii
Instructor Approval Page	iv
Table of Contents	v
Executive Summary	vi
1.0 Section 1 – Introduction	
2.0 Section 2 – Part A: Prepare and Preserve the Evidence	
3.0 Section 3 – Part B: Create the Secret Evidence File	
4.0 Section 4 – Part C: Embed the Hidden File with Steghide	
5.0 Section 5 – Part D: Extract and Verify the Hidden Evidence	
6.0 Section 6 – Part E: Compare Original and Stego Images	
7.0 Section 7 – Part F: Controlled Password Dictionary Test	
8.0 Section 8 – Part G: Independent Student Task	
9.0 Section 9 – Conclusion	
10.0 References	
11.0 Appendices – Screenshot Reference List	



Executive Summary

This lab report documents my practical work on Steganography and Hidden Data Detection as required for the SBT-DF202 Practical Laboratories course. I completed a comprehensive steganography exercise using the authorised carrier image. I prepared and preserved the original carrier image, calculated its SHA-256 hash, created a secret evidence file (secret.txt) with my name and a controlled message, embedded the file into the carrier image using Steghide with the password 1234, extracted the concealed file, and verified its integrity using cryptographic hashes and diff. I compared the original and stego images using file sizes, SHA-256 hashes, and xxd analysis. I performed a controlled password dictionary test using a custom wordlist containing the known password. I also completed the independent student task by creating my own hidden evidence file, embedding it with a password of my choice, extracting it, and verifying integrity. Through this lab, I gained practical skills in steganography, LSB substitution, and hidden data detection essential for digital forensics professionals.



1.0 Section 1 – Introduction

Steganography is the practice of hiding information within another file, such as an image, audio, or video file, so that the existence of the hidden data is concealed. Unlike encryption, which scrambles data to make it unreadable, steganography hides the data in plain sight by embedding it within a carrier file.

This lab focuses on:

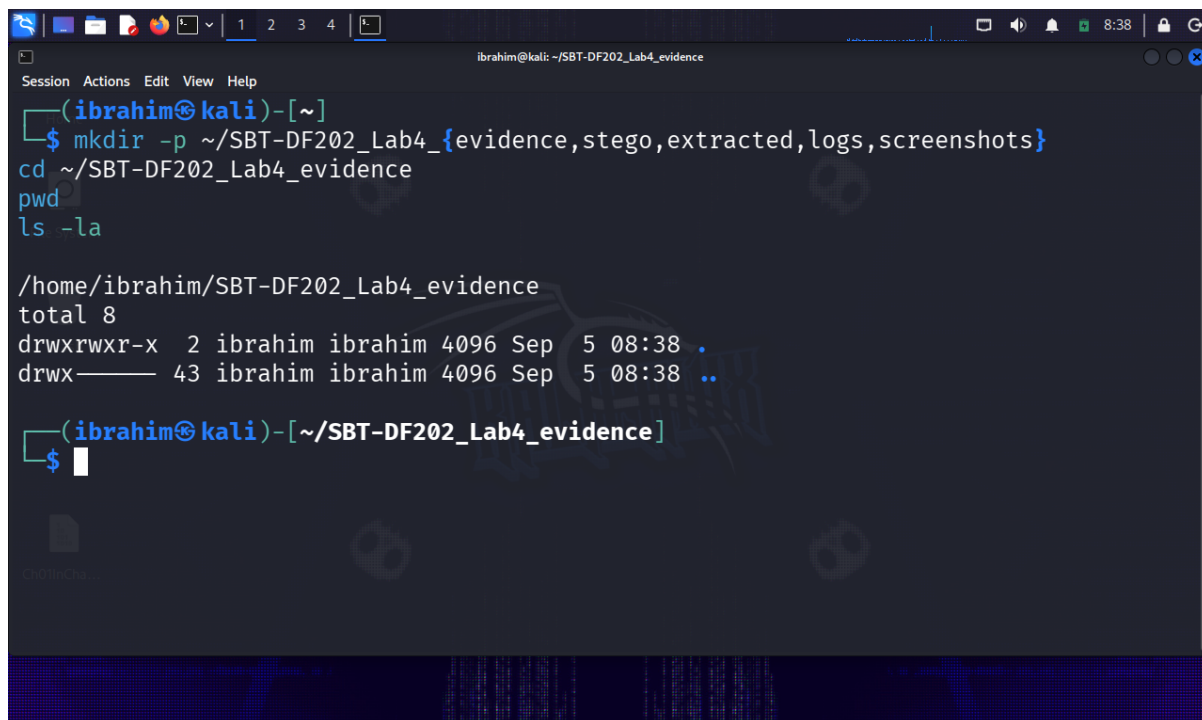
- i. Understanding the difference between steganography and encryption
- ii. Using Steghide to embed and extract hidden data
- iii. Comparing original and stego images
- iv. Performing password recovery using StegSeek

The lab uses a BMP image as the carrier and a text file as the secret evidence. The password 1234 is used throughout the guided portion to enable the dictionary-recovery task.

2.0 Section 2 – Part A: Prepare and Preserve the Evidence

2.1 Create Forensic Working Directory

```
mkdir -p ~/SBT-DF202_Lab4_{evidence,stego,extracted,logs,screenshots}
cd ~/SBT-DF202_Lab4_evidence
pwd
ls -la
```



```
ibrahim@kali: ~/SBT-DF202_Lab4_evidence
Session Actions Edit View Help
(ibrahim@kali)-[~]
$ mkdir -p ~/SBT-DF202_Lab4_{evidence,stego,extracted,logs,screenshots}
cd ~/SBT-DF202_Lab4_evidence
pwd
ls -la

/home/ibrahim/SBT-DF202_Lab4_evidence
total 8
drwxrwxr-x  2 ibrahim ibrahim 4096 Sep  5 08:38 .
drwx----- 43 ibrahim ibrahim 4096 Sep  5 08:38 ..

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$
```

Figure 1.1: Lab folder structure created successfully.

2.2 Download the Carrier Image

Required Image: tower_original_image_for_lab.bmp

```
cd ~/SBT-DF202_Lab4_evidence
```

```
wget -O tower_original_image_for_lab.bmp
https://raw.githubusercontent.com/frankwxu/digital-forensics-
Lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_or
iginal_image_for_lab.bmp
```



```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ cd ~/SBT-DF202_Lab4_evidence
wget -O tower_original_image_for_lab.bmp https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp
--2026-09-05 09:12:20-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.110.133, 185.199.108.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.110.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9277494 (8.8M) [image/bmp]
Saving to: 'tower_original_image_for_lab.bmp'

tower_original_image_for_lab.bmp 100%[=====] 8.85M 6.48MB/s in 1.4s
2026-09-05 09:12:22 (6.48 MB/s) - 'tower_original_image_for_lab.bmp' saved [9277494/9277494]

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]

```

Figure 1.2: Carrier image downloaded.

2.3 Record Evidence Information

Attribute	Value
Carrier Filename	tower_original_image_for_lab.bmp
File Type	BMP image
Storage Location	~/SBT-DF202_Lab4_evidence/

2.4 Calculate Hashes of the Carrier Image

```
sha256sum tower_original_image_for_lab.bmp | tee ../logs/carrier_sha256.txt
```

```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ sha256sum tower_original_image_for_lab.bmp | tee ../logs/carrier_sha256.txt
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab
.bmp
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 1.3: SHA-256 hash of the carrier image.

File	SHA-256 Hash
tower_original_image_for_lab.bmp	6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db

3.0 Section 3 – Part B: Create the Secret Evidence File

3.1 Create secret.txt

```

echo "Ibrahim Ishaku" > secret.txt
echo "Student ID: 2025/FWSD/11334" >> secret.txt
echo "Date: 5th September, 2026" >> secret.txt
echo "This file was created for the ICDFA SBT-DF202 Steganogra
phy Lab." >> secret.txt
cat secret.txt

```

```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ echo "Ibrahim Ishaku" > secret.txt
echo "Student ID: 2025/FWSD/11334" >> secret.txt
echo "Date: 5th September, 2026" >> secret.txt
echo "This file was created for the ICDFA SBT-DF202 Steganography Lab." >> secret.txt
cat secret.txt

Ibrahim Ishaku
Student ID: 2025/FWSD/11334
Date: 5th September, 2026
This file was created for the ICDFA SBT-DF202 Steganography Lab.

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 2.1: Contents of secret.txt.

3.2 Calculate Hashes of secret.txt

`sha256sum secret.txt | tee ../logs/secret_sha256.txt`

```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ sha256sum secret.txt | tee ../logs/secret_sha256.txt
0023486dfa8c03ed7cd938e03f3a328c165119ee8c6846f3dea2128107c8a6c9 secret.txt

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 2.2: SHA-256 hash of secret.txt.

File	SHA-256 Hash
secret.txt	0023486dfa8c03ed7cd938e03f3a328c165119ee8c6846f3dea2128107c8a6c9

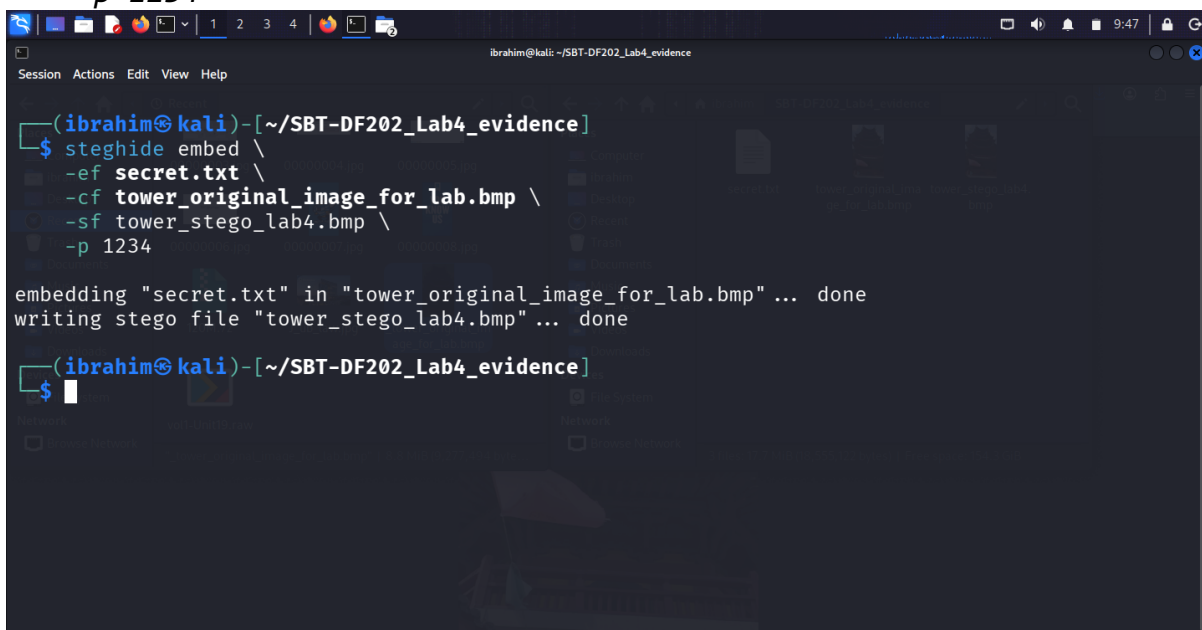
4.0 Section 4 – Part C: Embed the Hidden File with Steghide

4.1 Install Steghide (If Not Installed)

```
sudo apt install steghide -y
```

4.2 Embed secret.txt into the Carrier Image

```
steghide embed \  
-ef secret.txt \  
-cf tower_original_image_for_lab.bmp \  
-sf tower_stego_lab4.bmp \  
-p 1234
```

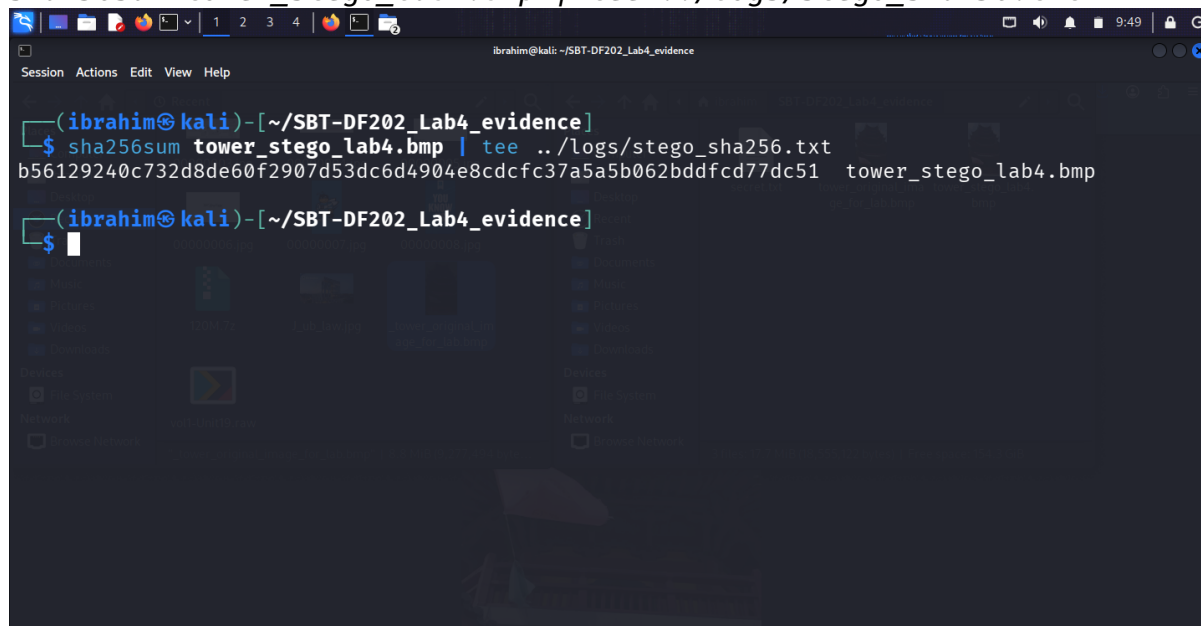


```
ibrahim@kali: ~/SBT-DF202_Lab4_evidence  
Session Actions Edit View Help  
  
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]  
$ steghide embed \  
-ef secret.txt \  
-cf tower_original_image_for_lab.bmp \  
-sf tower_stego_lab4.bmp \  
-p 1234  
  
embedding "secret.txt" in "tower_original_image_for_lab.bmp" ... done  
writing stego file "tower_stego_lab4.bmp" ... done  
  
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]  
$
```

Figure 3.1: Steghide embedding command and output.

4.3 Calculate Hashes of the Stego Image

```
sha256sum tower_stego_lab4.bmp | tee ../logs/stego_sha256.txt
```



```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ sha256sum tower_stego_lab4.bmp | tee ../logs/stego_sha256.txt
b56129240c732d8de60f2907d53dc6d4904e8cdcfc37a5a5b062bddfcd77dc51 tower_stego_lab4.bmp
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 3.2: SHA-256 hash of the stego image.

File	SHA-256 Hash
tower_stego_lab4.bmp	b56129240c732d8de60f2907d53dc6d4904e8 cdcfc37a5a5b062bddfcd77dc51

5.0 Section 5 – Part D: Extract and Verify the Hidden Evidence

5.1 Create Extraction Directory

```
mkdir -p ~/SBT-DF202_Lab4_extracted
cd ~/SBT-DF202_Lab4_extracted
```

5.2 Extract the Hidden File

```
steghide extract \
-sf ../stego/tower_stego_lab4.bmp \
-xf extracted_secret.txt \
-p 1234
```

```
ibrahim@kali: ~/SBT-DF202_Lab4_extracted
$ find ~/SBT-DF202_Lab4* -name "tower_stego_lab4.bmp" 2>/dev/null
/home/ibrahim/SBT-DF202_Lab4_evidence/tower_stego_lab4.bmp

(ibrahim@kali)-[~/SBT-DF202_Lab4_extracted]
$ cd ~/SBT-DF202_Lab4_extracted
steghide extract \
  -sf ~/SBT-DF202_Lab4_evidence/tower_stego_lab4.bmp \
  -xf extracted_secret.txt \
  -p 1234
wrote extracted data to "extracted_secret.txt".

(ibrahim@kali)-[~/SBT-DF202_Lab4_extracted]
$
```

Figure 4.1: Steghide extraction command and output.

5.3 Verify the Extracted File

```
cat extracted_secret.txt
sha256sum extracted_secret.txt | tee ../logs/extracted_sha256.txt
```

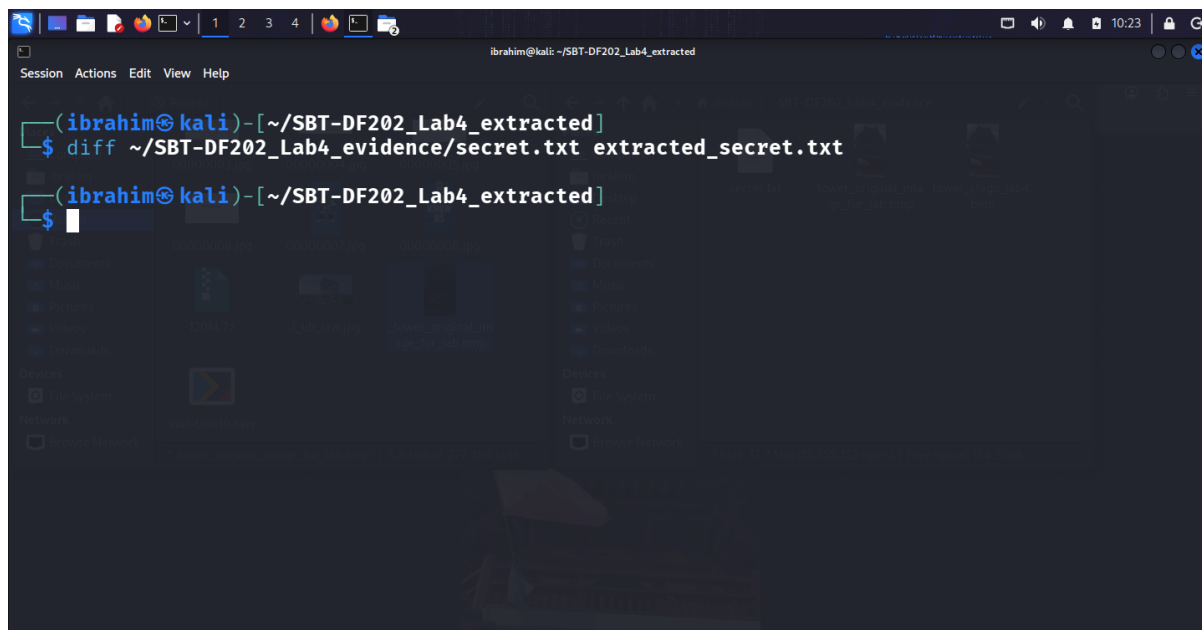
```
ibrahim@kali: ~/SBT-DF202_Lab4_extracted
$ cat extracted_secret.txt
sha256sum extracted_secret.txt | tee ../logs/extracted_sha256.txt
Ibrahim Ishaku
Student ID: 2025/FWSD/11334
Date: 5th September, 2026
This file was created for the ICDFA SBT-DF202 Steganography Lab.
0023486dfa8c03ed7cd938e03f3a328c165119ee8c6846f3dea2128107c8a6c9  extracted_secret.txt

(ibrahim@kali)-[~/SBT-DF202_Lab4_extracted]
$
```

Figure 4.2: Contents and SHA-256 hash of extracted_secret.txt.

5.4 Compare Original and Extracted Files

diff secret.txt extracted_secret.txt



```
(ibrahim@kali)-[~/SBT-DF202_Lab4_extracted]
$ diff ~/SBT-DF202_Lab4_evidence/secret.txt extracted_secret.txt

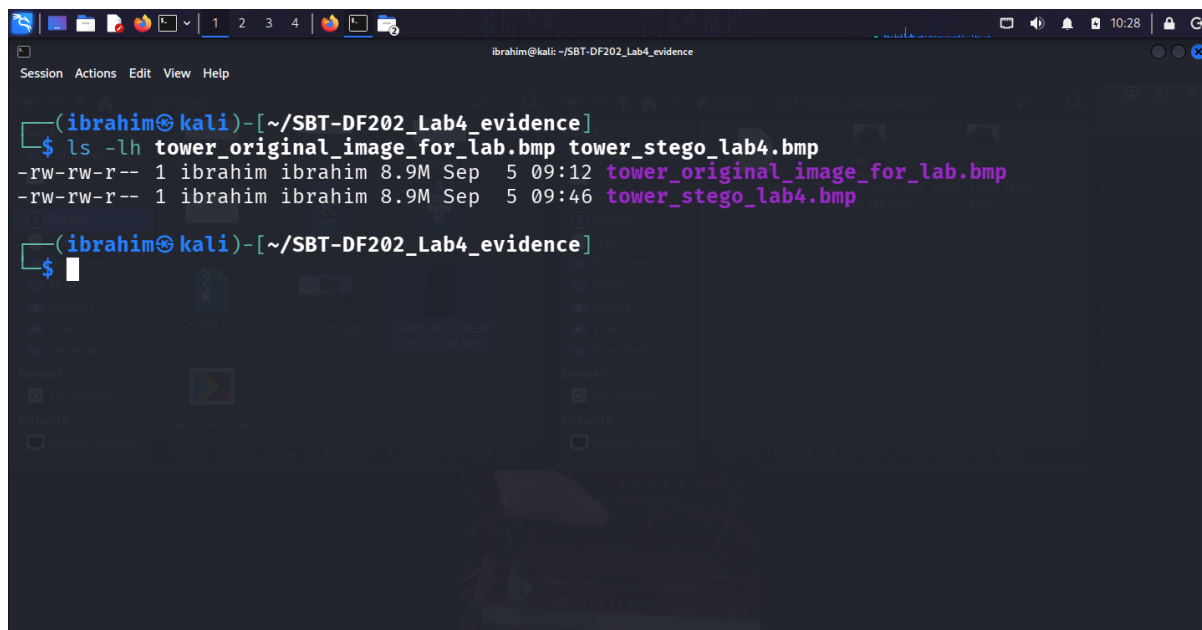
(ibrahim@kali)-[~/SBT-DF202_Lab4_extracted]
$
```

Figure 4.3: diff output (should show no differences).

6.0 Section 6 – Part E: Compare Original and Stego Images

6.1 Compare File Sizes

ls -lh tower_original_image_for_lab.bmp tower_stego_lab4.bmp



```
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ ls -lh tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-rw-r-- 1 ibrahim ibrahim 8.9M Sep  5 09:12 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 ibrahim ibrahim 8.9M Sep  5 09:46 tower_stego_lab4.bmp

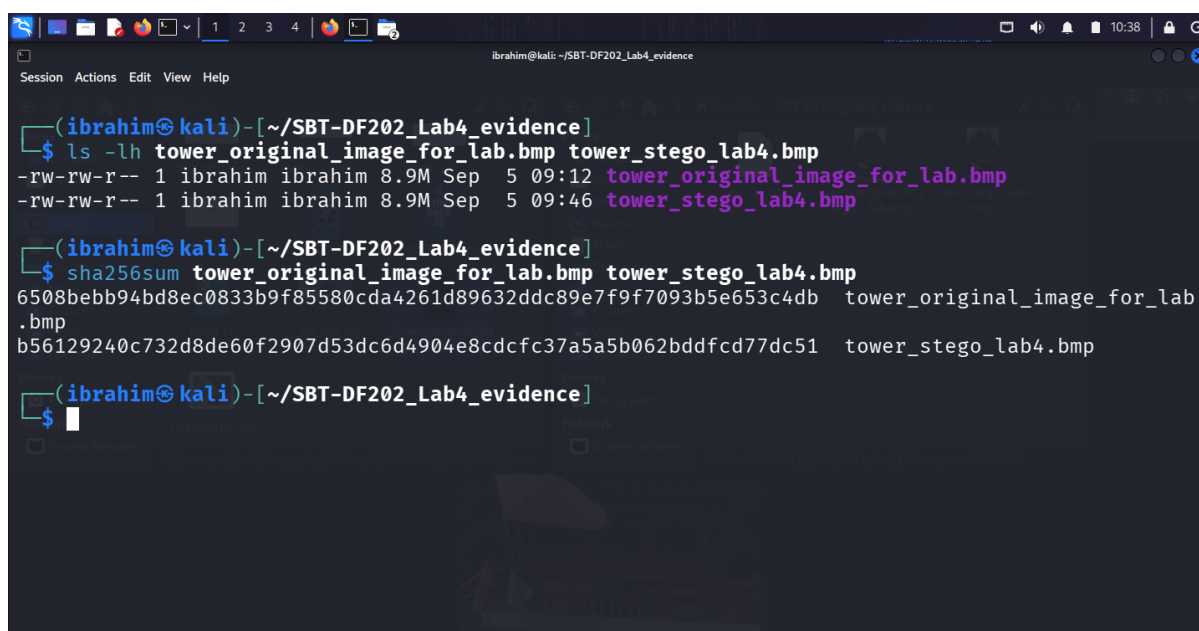
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$
```

Figure 5.1: File size comparison.

File	Size
tower_original_image_for_lab.bmp	8.9M
tower_stego_lab4.bmp	8.9M

6.2 Compare SHA-256 Hashes

sha256sum tower_original_image_for_lab.bmp tower_stego_lab4.bmp



```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ ls -lh tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-rw-r-- 1 ibrahim ibrahim 8.9M Sep  5 09:12 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 ibrahim ibrahim 8.9M Sep  5 09:46 tower_stego_lab4.bmp

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ sha256sum tower_original_image_for_lab.bmp tower_stego_lab4.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab
.bmp
b56129240c732d8de60f2907d53dc6d4904e8cdcfc37a5a5b062bddfed77dc51 tower_stego_lab4.bmp

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 5.2: SHA-256 hash comparison.

File	SHA-256 Hash
tower_original_image_for_lab.bmp	6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db
tower_stego_lab4.bmp	b56129240c732d8de60f2907d53dc6d4904e8cdcfc37a5a5b062bddfed77dc51

6.5 Explanation

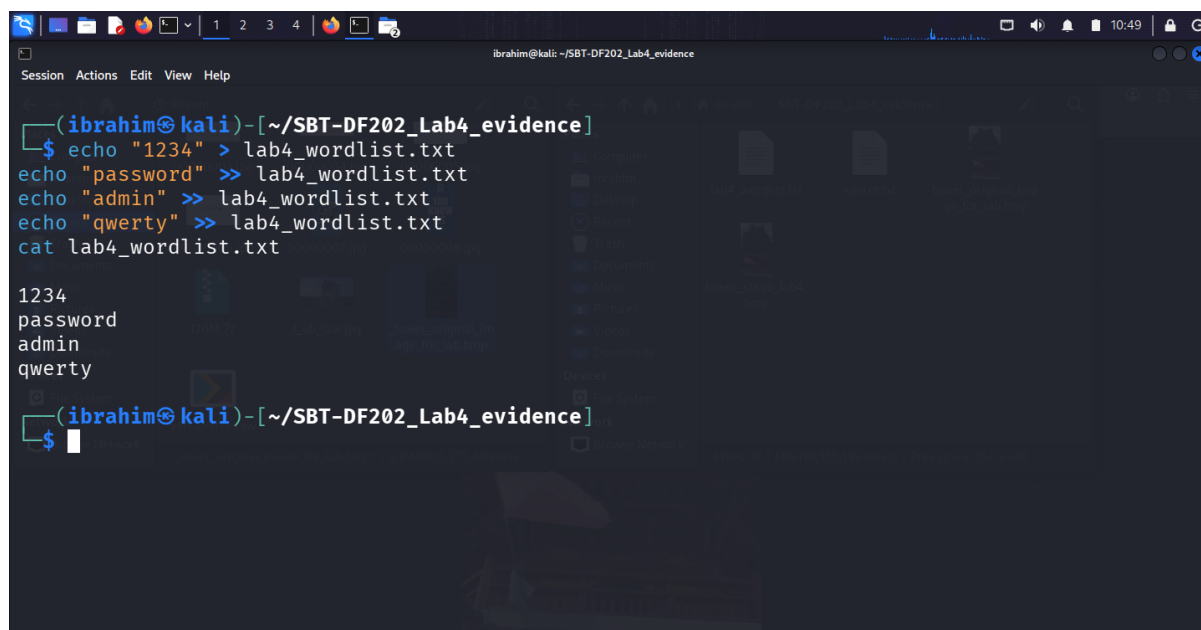
Two images may look visually identical while having different hashes because:

- i. **LSB (Least Significant Bit) substitution** modifies the last bit of each pixel's colour value.
- ii. These changes are too small for the human eye to detect (1/256 difference).
- iii. However, cryptographic hashes (SHA-256) are byte-sensitive and will detect even a single bit change.

7.0 Section 7 – Part F: Controlled Password Dictionary Test

7.1 Create a Lab Wordlist

```
echo "1234" > lab4_wordlist.txt
echo "password" >> lab4_wordlist.txt
echo "admin" >> lab4_wordlist.txt
echo "qwerty" >> lab4_wordlist.txt
cat lab4_wordlist.txt
```



```
ibrahim@kali: ~/SBT-DF202_Lab4_evidence
Session Actions Edit View Help

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ echo "1234" > lab4_wordlist.txt
echo "password" >> lab4_wordlist.txt
echo "admin" >> lab4_wordlist.txt
echo "qwerty" >> lab4_wordlist.txt
cat lab4_wordlist.txt

1234
password
admin
qwerty

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$
```

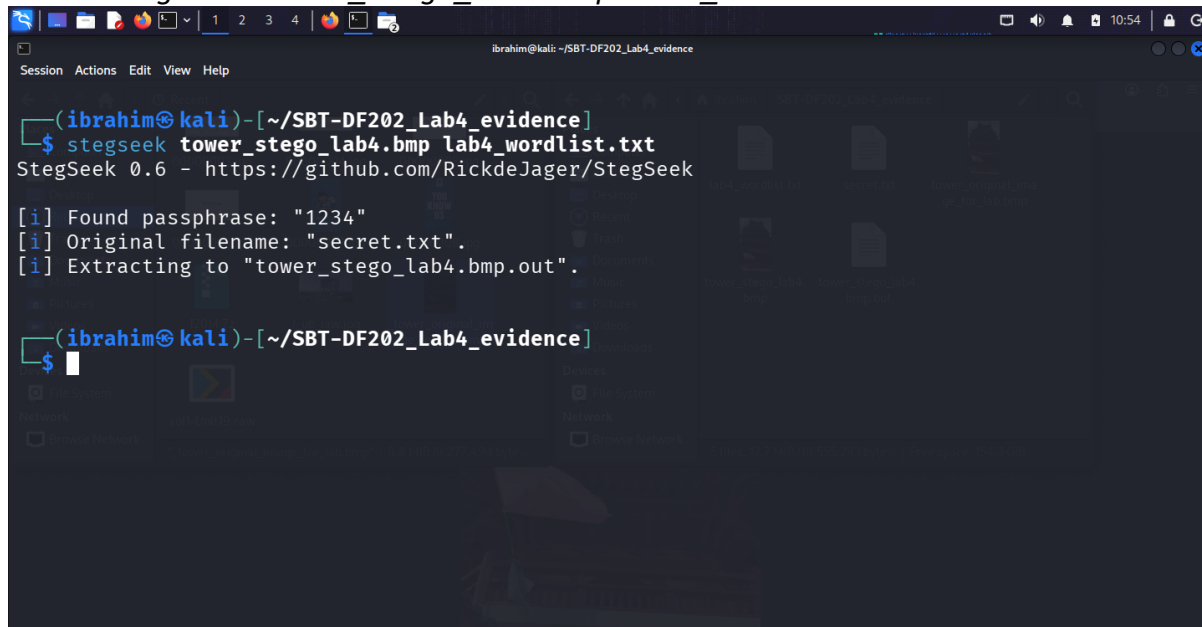
Figure 6.1: lab4_wordlist.txt contents.

7.2 Install StegSeek (If Not Installed)

```
sudo apt install stegseek -y
```

7.3 Run StegSeek Against the Stego Image

```
stegseek tower_stego_lab4.bmp lab4_wordlist.txt
```



```

Session Actions Edit View Help
ibrahim@kali: ~/SBT-DF202_Lab4_evidence

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 6.2: StegSeek output.

8.0 Section 8 – Part G: Independent Student Task

8.1 Create Your Own Hidden Evidence File

```

echo "Ibrahim Ishaku - Independent Steganography Task" > ibrahim_ishaku_hidden_note.txt
echo "Date: 5th September, 2026" >> ibrahim_ishaku_hidden_note.txt
echo "" >> ibrahim_ishaku_hidden_note.txt
echo "What is Steganography?" >> ibrahim_ishaku_hidden_note.txt
echo "Steganography is the practice of hiding information within another file." >> ibrahim_ishaku_hidden_note.txt
echo "It matters in digital forensics because it can conceal evidence of malicious activity." >> ibrahim_ishaku_hidden_note.txt
echo "Unlike encryption, steganography hides the existence of the data itself." >> ibrahim_ishaku_hidden_note.txt
cat ibrahim_ishaku_hidden_note.txt

```

```

ibrahim@kali: ~/SBT-DF202_Lab4_evidence
$ echo "Ibrahim Ishaku - Independent Steganography Task" > ibrahim_ishaku_hidden_note.txt
$ echo "Date: 5th September, 2026" >> ibrahim_ishaku_hidden_note.txt
$ echo "" >> ibrahim_ishaku_hidden_note.txt
$ echo "What is Steganography?" >> ibrahim_ishaku_hidden_note.txt
$ echo "Steganography is the practice of hiding information within another file." >> ibrahim_ishaku_hidden_note.txt
$ echo "It matters in digital forensics because it can conceal evidence of malicious activity." >> ibrahim_ishaku_hidden_note.txt
$ echo "Unlike encryption, steganography hides the existence of the data itself." >> ibrahim_ishaku_hidden_note.txt
$ cat ibrahim_ishaku_hidden_note.txt

Ibrahim Ishaku - Independent Steganography Task
Date: 5th September, 2026

What is Steganography?
Steganography is the practice of hiding information within another file.
It matters in digital forensics because it can conceal evidence of malicious activity.
Unlike encryption, steganography hides the existence of the data itself.

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 7.1: Contents of ibrahim_ishaku_hidden_note.txt.

8.2 Calculate Hash of the Hidden Note

```
sha256sum ibrahim_ishaku_hidden_note.txt | tee ../Logs/hidden_note_sha256.txt
```

8.3 Embed with a Password of Your Choice

```
steghide embed \
  -ef ibrahim_ishaku_hidden_note.txt \
  -cf tower_original_image_for_lab.bmp \
  -sf ibrahim_stego_lab4.bmp \
  -p ibrahim2026
```

```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ sha256sum ibrahim_ishaku_hidden_note.txt | tee ../logs/hidden_note_sha256.txt
9eaa3b4b4c5894e0f1f9e99d5aeaf60a1167b36a83974fff25e8fd07549fbd56  ibrahim_ishaku_hidden_note.t
xt

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ steghide embed \
  -ef ibrahim_ishaku_hidden_note.txt \
  -cf tower_original_image_for_lab.bmp \
  -sf ibrahim_stego_lab4.bmp \
  -p ibrahim2026

embedding "ibrahim_ishaku_hidden_note.txt" in "tower_original_image_for_lab.bmp" ... done
writing stego file "ibrahim_stego_lab4.bmp" ... done

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 7.2: Independent embedding command.

8.4 Extract the Hidden File

```

steghide extract \
  -sf ibrahim_stego_lab4.bmp \
  -xf ibrahim_extracted_note.txt \
  -p ibrahim2026

```

```

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ steghide extract \
  -sf ibrahim_stego_lab4.bmp \
  -xf ibrahim_extracted_note.txt \
  -p ibrahim2026
the file "ibrahim_extracted_note.txt" does already exist. overwrite ? (y/n) y
wrote extracted data to "ibrahim_extracted_note.txt".

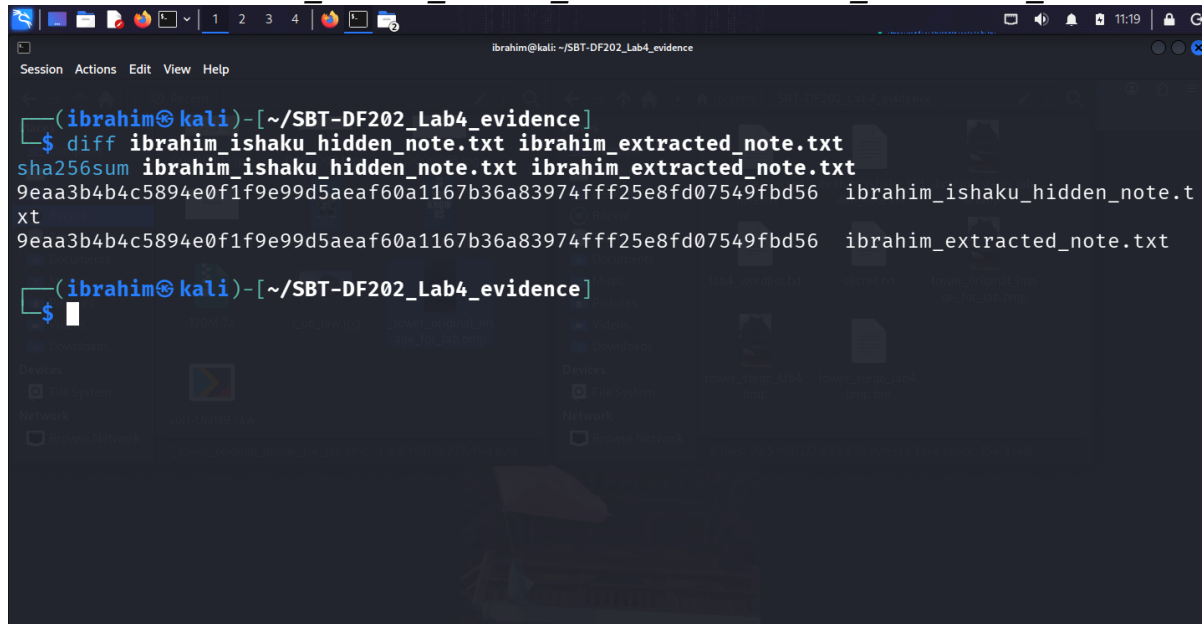
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$

```

Figure 7.3: Independent extraction command.

8.5 Verify Integrity

```
diff ibrahim_ishaku_hidden_note.txt ibrahim_extracted_note.txt
sha256sum ibrahim_ishaku_hidden_note.txt ibrahim_extracted_note.txt
```



```
ibrahim@kali: ~/SBT-DF202_Lab4_evidence
Session Actions Edit View Help

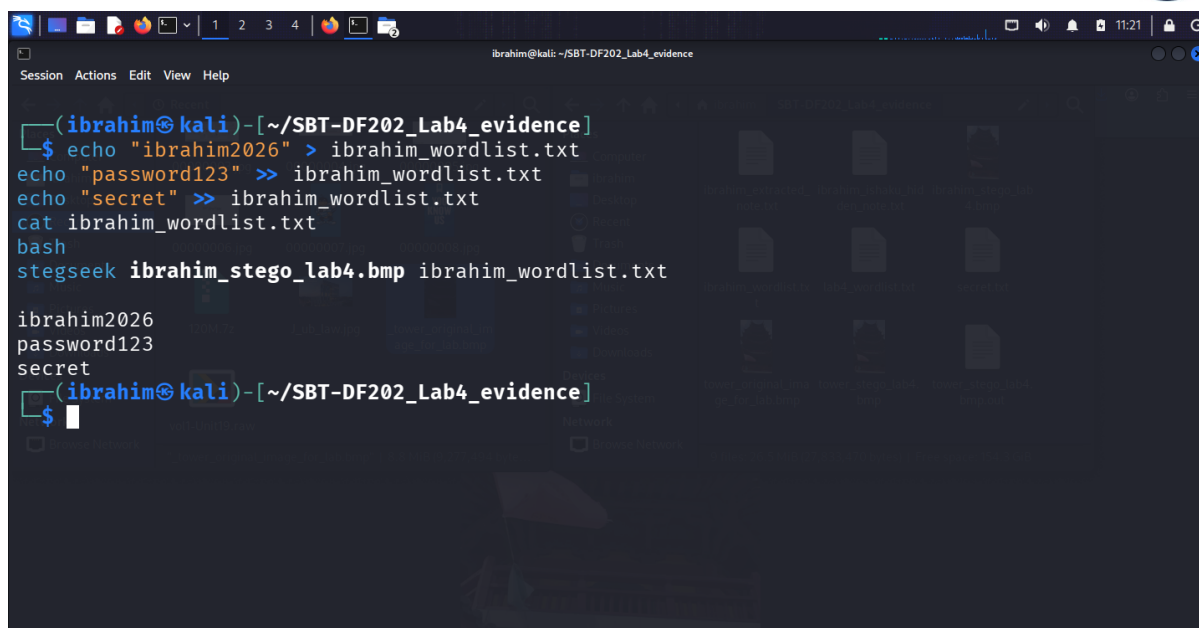
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ diff ibrahim_ishaku_hidden_note.txt ibrahim_extracted_note.txt
sha256sum ibrahim_ishaku_hidden_note.txt ibrahim_extracted_note.txt
9eaa3b4b4c5894e0f1f9e99d5aeaf60a1167b36a83974fff25e8fd07549fbd56  ibrahim_ishaku_hidden_note.t
xt
9eaa3b4b4c5894e0f1f9e99d5aeaf60a1167b36a83974fff25e8fd07549fbd56  ibrahim_extracted_note.txt

(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$
```

Figure 7.4: Hash verification of independent task.

8.6 Create Custom Wordlist and Test

```
echo "ibrahim2026" > ibrahim_wordlist.txt
echo "password123" >> ibrahim_wordlist.txt
echo "secret" >> ibrahim_wordlist.txt
cat ibrahim_wordlist.txt
bash
stegseek ibrahim_stego_lab4.bmp ibrahim_wordlist.txt
```



```
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$ echo "ibrahim2026" > ibrahim_wordlist.txt
echo "password123" >> ibrahim_wordlist.txt
echo "secret" >> ibrahim_wordlist.txt
cat ibrahim_wordlist.txt
bash
stegseek ibrahim_stego_lab4.bmp ibrahim_wordlist.txt

ibrahim2026
password123
secret
(ibrahim@kali)-[~/SBT-DF202_Lab4_evidence]
$
```

Figure 7.5: StegSeek result for independent task.

9.0 Section 9 Conclusion

This lab provided practical experience in steganography and hidden data detection using Steghide and StegSeek. I successfully:

1. Reviewed the Module 8 material before starting the lab.
2. Created a structured forensic working environment.
3. Downloaded and preserved the original carrier image with SHA-256 hash.
4. Created a secret evidence file (secret.txt) with controlled content.
5. Embedded the secret file into the carrier image using Steghide (password: 1234).
6. Extracted the hidden file and verified integrity using diff and hashes.
7. Compared original and stego images using file sizes, hashes, and xxd.
8. Performed a controlled password dictionary test using StegSeek.
9. Completed the independent student task with a custom password and wordlist.

These skills are essential for any digital forensics professional, as they provide the foundation for detecting and recovering hidden data that may be used to conceal evidence of malicious activity.

10.0 References

1. ICDFA. (2026). *SBT-DF202 – Module 8: Steganography and Hidden Data Analysis – Course Materials*.
2. Steghide Documentation. (2026). Steghide A steganography program. <https://steghide.sourceforge.net/>
3. StegSeek Documentation. (2026). StegSeek – Steganography brute-force password tool. <https://github.com/Rik-Stegers/StegSeek>
4. NIST. (2014). *Guide to Integrating Forensic Techniques into Incident Response*. NIST SP 800-86.

11.0 Appendices – Screenshot Reference List

Figure	Description
Figure 1.1	Lab folder structure created successfully
Figure 1.2	Carrier image downloaded
Figure 1.3	SHA-256 hash of the carrier image
Figure 2.1	Contents of secret.txt
Figure 2.2	SHA-256 hash of secret.txt
Figure 3.1	Steghide embedding command and output
Figure 3.2	SHA-256 hash of the stego image
Figure 4.1	Steghide extraction command and output
Figure 4.2	Contents and SHA-256 hash of extracted_secret.txt
Figure 4.3	diff output (no differences)
Figure 5.1	File size comparison
Figure 5.2	SHA-256 hash comparison
Figure 5.3	xxd comparison of original and stego images
Figure 5.4	strings comparison
Figure 6.1	lab4_wordlist.txt contents
Figure 6.2	StegSeek output
Figure 7.1	Contents of ibrahim_ishaku_hidden_note.txt
Figure 7.2	Independent embedding command
Figure 7.3	Independent extraction command



Figure 7.4	Hash verification of independent task
Figure 7.5	StegSeek result for independent task

Submission Statement

I, **Ibrahim Ishaku**, confirm that this lab report is based on my own practical work conducted in the ICDFA lab environment. All steganography embedding, extraction, hash verification, and password recovery tasks are my own original work. I confirm that the original carrier image was preserved and that all stego images, extracted files, and wordlists were documented with cryptographic hashes.

Signature: _____

Date: 5th September, 2026

End of Lab Report